实验四：图神经网络（GNN）

- 实验四：图神经网络（GNN）
  - 概述
    - 实现内容
  - 环境配置
  - 任务总览与推荐完成顺序
  - Task 1：图的表示与邻接矩阵
    - 步骤 1.1：`make_adjacency`
    - 步骤 1.2：`add_self_loops`
    - 步骤 1.3：`degree_vector`
    - 步骤 1.4：`row_normalize` 与 `symmetric_normalize`
    - 步骤 1.5：`graph_laplacian`
    - 步骤 1.6：`dirichlet_energy` 与 `propagate`
    - 检查点 A
  - Task 2：GCN 图卷积层
    - 步骤 2.1：`aggregate` 与 `aggregate_backward`
    - 步骤 2.2：`graph_conv_forward_naive`
    - 步骤 2.3：`graph_conv_forward`
    - 步骤 2.4：`graph_conv_backward`
    - 检查点 B
  - Task 3：GraphSAGE 与通用聚合器
    - 步骤 3.1：`max_aggregate` 与 `max_aggregate_backward`
    - 步骤 3.2：`sage_conv_forward` 与 `sage_conv_backward`
    - 检查点 C
  - Task 4：半监督节点分类模型
    - 4.1 半监督掩码损失
    - 4.2 组装 `GraphNet` 类
      - 步骤 4.1：`_conv` / `_conv_backward` / `_conv_relu` / `_conv_relu_backward`
      - 步骤 4.2：`loss` 与 `_backward`
      - 步骤 4.3：`predict`
    - 检查点 D
  - Task 5：训练循环与 Cora 实战
    - 5.1 训练循环（`src/train.py`）
      - 步骤 5.1：`evaluate`
      - 步骤 5.2：`train`
    - 检查点 E：全部通过
    - 5.2 数据集与预处理
    - 5.3 正式训练与调参（报告内容）
    - 你在实验报告里面需要写什么?
  - 提供的代码（无需修改）
  - 评分标准
    - 第一部分：自动化测试（80 分）
    - 第二部分：Cora 训练与报告（20 分）
    - 提交要求
  - 参考资料

概述

图数据由节点、节点特征与边组成，社交网络、论文引用、分子结构和知识图谱都是常见的图结构数据。与规则网格不同，图中每个节点的邻居数量可能不同。图神经网络（Graph Neural Network, GNN）通过消息传递处理这种不规则结构：每个节点从邻居收集信息（聚合，aggregate），再与自身信息合并并做可学习的变换（更新，update）。堆叠 L 层后，信息可以沿边传播 L 跳。这一视角由 Gilmer 等人统一表述[6]，本实验将实现其中两个经典模型：GCN[1] 与 GraphSAGE[2]。

本次实验的目标是：只用 NumPy，从零实现图卷积层与消息传递聚合器，搭出 GCN / GraphSAGE 模型，并在真实的 Cora 论文引用网络上完成半监督节点分类——每类只看 20 篇有标签的论文（共 140 个节点，约占 5%），对其余 1000 篇论文的学科类别做出预测。

实现内容

按依赖关系，你需要依次完成以下五个部分。前面的部分是后面的地基，请务必按顺序完成，不要跳步：

顺序	部分	内容	所在文件
①	Task 1	图的表示与邻接矩阵：`make_adjacency` → `add_self_loops` → `degree_vector` → `row_normalize` / `symmetric_normalize` → `graph_laplacian` → `dirichlet_energy` → `propagate`	`src/graph.py`
②	Task 2	GCN 图卷积层：`aggregate` → `aggregate_backward` → `graph_conv_forward_naive` → `graph_conv_forward` → `graph_conv_backward`	`src/gcn.py`
③	Task 3	GraphSAGE 与聚合器：`max_aggregate` → `max_aggregate_backward` → `sage_conv_forward` → `sage_conv_backward`	`src/sage.py`
④	Task 4	半监督模型：`masked_softmax_loss` → `masked_accuracy` → `GraphNet` 类（前向/反向/预测）	`src/model.py`
⑤	Task 5	训练循环：`evaluate` → `train`，然后在 Cora 上正式训练、做三组对照实验并写报告	`src/train.py`、`train_cora.py`

> 本实验直接使用已提供的通用层与优化器（`src/layers.py`、`src/optim.py`），无需修改。

环境配置

请首先安装 `uv` Python 项目管理工具：

```
# On macOS and Linux.
curl -LsSf https://astral.sh/uv/install.sh | sh
```

```
# On Windows.
powershell -ExecutionPolicy ByPass -c "irm https://astral.sh/uv/install.ps1 | iex"
```

先确认安装完毕，然后在项目根目录下执行：

```
uv sync
```

该指令会在根目录下创建 Python 虚拟环境。本实验使用 `scipy` 存储 Cora 的稀疏邻接矩阵，并使用 `networkx` 绘制图结构。若使用 IDE，可将解释器设置为该虚拟环境中的解释器。

之后可以尝试运行测试用例（路径中不可带非 ascii 编码字符）：

```
uv run pytest
```

若输出类似以下的内容，则说明环境配置完毕：

```
...FFFF
```

任务总览与推荐完成顺序

下面这张表是整个实验的「地图」。第 3 列「依赖」告诉你必须先完成什么；请严格按照从上到下的顺序推进，每完成一行就做一次「检查点」确认，再进入下一行。

步骤	函数/方法	依赖	完成后跑什么验证	所在文件
1.1	`make_adjacency`	无	3 个 `make_adjacency` 测试	`src/graph.py`
1.2	`add_self_loops`	1.1	2 个 `add_self_loops` 测试	`src/graph.py`
1.3	`degree_vector`	1.1	`test_degree_vector_values`	`src/graph.py`
1.4	`row_normalize` / `symmetric_normalize`	1.3	4 个归一化测试	`src/graph.py`
1.5	`graph_laplacian`	1.3	2 个 `graph_laplacian` 测试	`src/graph.py`
1.6	`dirichlet_energy` / `propagate`	1.5、1.4	`test_dirichlet_energy` 、 `test_propagate`	`src/graph.py`
1	检查点 A：图工具全部通过	—	`uv run pytest tests/test_graph.py`（14/14）	—
2.1	`aggregate` / `aggregate_backward`	1.4	4 个 `aggregate` 测试	`src/gcn.py`
2.2	`graph_conv_forward_naive`	1.2	`test_graph_conv_forward_naive_hand_computed`	`src/gcn.py`
2.3	`graph_conv_forward`	2.1、2.2	`matches_naive` 、 `shapes` 、 `bias` 等	`src/gcn.py`
2.4	`graph_conv_backward`	2.3	2 个数值梯度测试 + 2 个感受野测试	`src/gcn.py`
2	检查点 B：GCN 层全部通过	—	`uv run pytest tests/test_gcn.py`（13/13）	—
3.1	`max_aggregate` / `max_aggregate_backward`	1.1	4 个 `max_aggregate` 测试	`src/sage.py`
3.2	`sage_conv_forward` / `sage_conv_backward`	2.1、3.1	7 个 `sage` 测试	`src/sage.py`
3	检查点 C：GraphSAGE 全部通过	—	`uv run pytest tests/test_sage.py`（11/11）	—
4.1	`masked_softmax_loss` / `masked_accuracy`	无	4 个 `masked` 测试	`src/model.py`
4.2	`GraphNet.loss` / `_backward` / `predict`	检查点 B、C（或至少 B）	数值梯度与前向形状测试	`src/model.py`
4	检查点 D：模型测试通过	—	`uv run pytest tests/test_model.py`（12/12）	—
5.1	`evaluate` / `train`	检查点 D	`uv run pytest tests/test_train.py`（8/8）	`src/train.py`
5	检查点 E：全部 58 例通过	—	`uv run pytest`（58/58）	—

Task 1：图的表示与邻接矩阵

一张图 $G = (V, E)$ 由 N 个节点和若干条边组成。本实验只考虑无向无权图，用邻接矩阵（adjacency matrix）$A \in \{0,1\}^{N\times N}$ 表示：$A_{ij} = 1$ 当且仅当节点 i 与 j 之间有边。节点 i 的度数（degree）$d_i = \sum_j A_{ij}$ 即它的邻居数量。

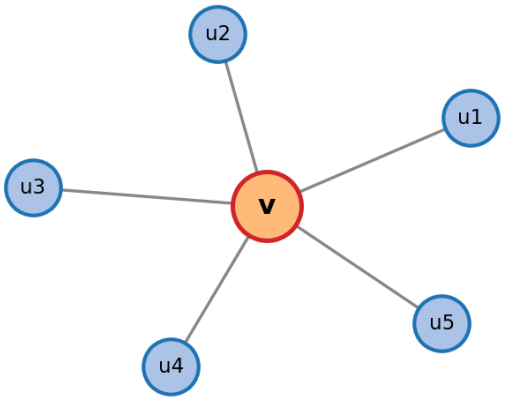
GCN 的每一层是

$$H^{(l+1)} = \sigma\left(\hat{A}\, H^{(l)} W^{(l)}\right), \qquad \hat{A} = \tilde{D}^{-1/2}\,\tilde{A}\,\tilde{D}^{-1/2}, \quad \tilde{A} = A + I$$

其中 $H^{(l)}$ 的第 i 行是节点 i 的特征向量，$\tilde{D}$ 是 $\tilde{A}$ 的度数对角阵[1]。拆开看，$\hat{A}H$ 这一步就是「每个节点把邻居的特征按权重求和」，权重 $\hat{A}_{ij} = 1/\sqrt{\tilde{d}_i \tilde{d}_j}$。这里有两个设计决策，本任务就是把它们逐一实现出来：

1. 自环（self loop）：$\tilde{A} = A + I$ 让每个节点在聚合时把自己也算作邻居。没有自环，节点的新表示将完全由邻居决定，自身的特征信息每过一层就被丢掉一次。
2. 归一化（normalization）：直接用 A 聚合，度数大的节点会收到巨大的特征和（Cora 上度数从 1 到 168 不等），数值尺度随度数爆炸。对称归一化 $\tilde{D}^{-1/2}\tilde{A}\tilde{D}^{-1/2}$ 把每条边的权重压到 $1/\sqrt{\tilde{d}_i\tilde{d}_j}$，保证结果的谱半径不超过 1——这让多层堆叠在数值上稳定。另一种常见选择是行归一化 $D^{-1}A$，聚合变成邻居特征的简单平均（随机游走视角）。

(a) message passing



aggregate messages from neighbours

(b) one GCN layer

$$H^{(l+1)} = \sigma\left(\hat{A}\, H^{(l)} W^{(l)}\right)$$

$$\hat{A} = \tilde{D}^{-1/2}\, \tilde{A}\, \tilde{D}^{-1/2}, \quad \tilde{A} = A + I$$

$$h_v^{(l+1)} = \sigma\left(\sum_{u \in \mathcal{N}(v) \cup \{v\}} \hat{A}_{vu}\, W^{(l)\top} h_u^{(l)} \right)$$

> 这一整节只允许使用 NumPy，不得调用任何自动微分工具。

步骤 1.1：`make_adjacency`

目标：从边列表构建对称的 0-1 邻接矩阵。这是数据进入模型的第一道门。

函数签名：

```
def make_adjacency(num_nodes, edges):
    # num_nodes: 节点数 N
    # edges: (E, 2) 整数数组，每行一条边 (i, j)
    # 返回 A: (N, N) float64，取值 {0, 1}，对角线为 0
```

实现要求：图视为无向——每条边同时置 `A[i, j]` 与 `A[j, i]`；重复边与自环边（`i == j`）忽略。

验证：

```
uv run pytest tests/test_graph.py -k make_adjacency
```

应看到 3 passed。

步骤 1.2：`add_self_loops`

目标：返回 $\tilde{A} = A + I$。

函数签名：

```
def add_self_loops(A):
    # 返回新数组，不得修改输入 A
```

验证：

```
uv run pytest tests/test_graph.py -k add_self_loops
```

应看到 2 passed。

步骤 1.3：`degree_vector`

函数签名：

```
def degree_vector(A):
    # 返回 d: (N,) float64，d[i] = A[i].sum()
```

验证：

```
uv run pytest tests/test_graph.py -k degree_vector
```

应看到 1 passed。

步骤 1.4：`row_normalize` 与 `symmetric_normalize`

目标：实现两种邻接矩阵归一化。

$$D^{-1}A \quad \text{（行归一化）}, \qquad D^{-1/2}AD^{-1/2} \quad \text{（对称归一化）}$$

函数签名：

```
def row_normalize(A):
    # 返回新 (N, N) float64 数组；度数为 0 的孤立节点行保持全 0

def symmetric_normalize(A):
    # 返回新 (N, N) float64 数组；孤立节点同样处理
```

提示：对称归一化可以写成 `d_inv_sqrt[:, None] * A * d_inv_sqrt[None, :]`；孤立节点（$d_i = 0$）注意不要出现除零。

验证：

```
uv run pytest tests/test_graph.py -k "row_normalize or symmetric_normalize"
```

应看到 4 passed。其中 `test_symmetric_normalize_is_symmetric_and_spectral_radius_at_most_one` 检查输出仍对称且特征值绝对值不超过 1——这正是堆叠多层不会数值爆炸的原因。

步骤 1.5：`graph_laplacian`

目标：组合图拉普拉斯 $L = D - A$。

原理：对任意节点信号 $x \in \mathbb{R}^N$，有

$$x^\top L x = \sum_{(i,j)\in E} (x_i - x_j)^2 \geq 0$$

即 L 衡量信号在图上「有多不平滑」：相连的节点取值差异越大，二次型越大。L 的特征向量是「图上的傅里叶基」，GCN 最早正是从谱图理论（在 L 的特征基上做滤波）推导出来的[1]。本实验走空间域（消息传递）路线，但拉普拉斯在 Task 1.6 的分析工具里会用到。

函数签名：

```
def graph_laplacian(A):
    # 返回 L = D - A，(N, N) float64
```

验证：

```
uv run pytest tests/test_graph.py -k graph_laplacian
```

应看到 2 passed（其中 `test_graph_laplacian_quadratic_form` 验证上面的二次型恒等式）。

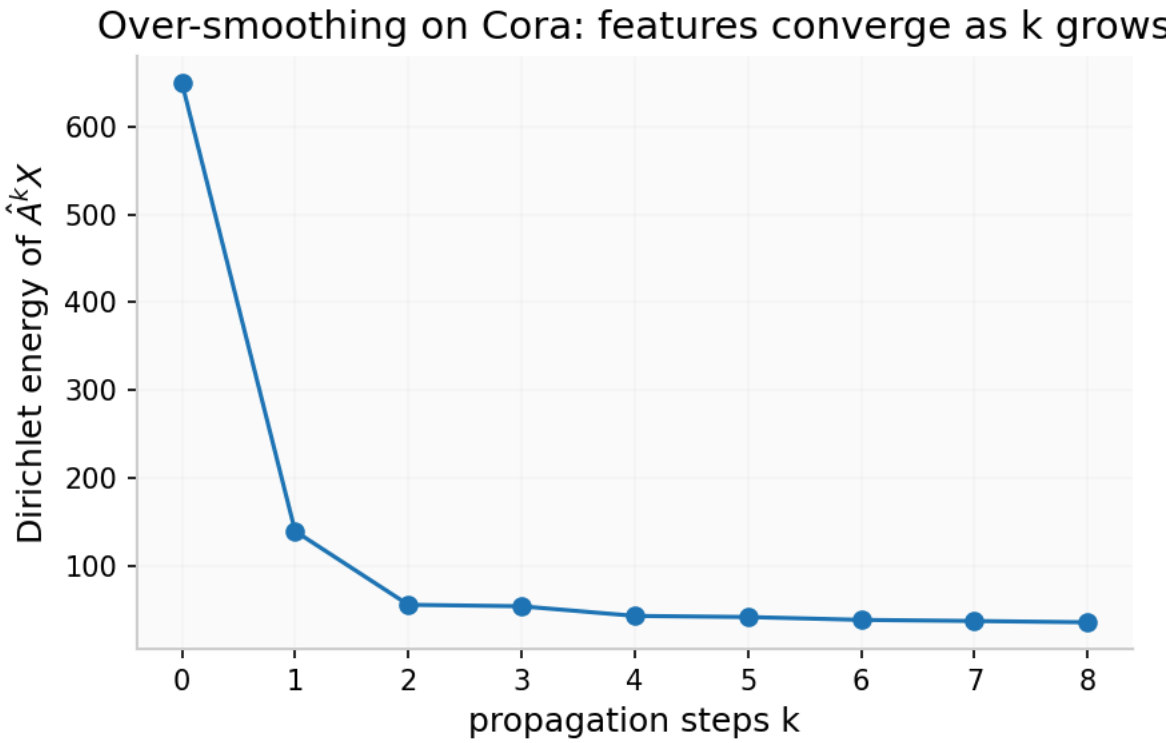
步骤 1.6：`dirichlet_energy` 与 `propagate`

目标：实现两个分析工具，报告中的过平滑实验会用到。

Dirichlet 能量：把上面的二次型推广到多通道特征 $X \in \mathbb{R}^{N\times F}$：

$$E(X) = \mathrm{tr}(X^\top L X) = \sum_{(i,j)\in E} \|x_i - x_j\|_2^2$$

多层传播：`propagate(A_norm, X, k)` 返回 $\hat{A}^k X$——相当于一个没有权重、不做学习的 k 层 GCN（这正是 SGC[8] 的视角）。观察 $E(\hat{A}^k X)$ 随 k 的变化，你就能在训练任何模型之前直接看到过平滑（over-smoothing）：特征沿边不断被平均，节点之间越来越像。在真实 Cora 上测量，这条曲线长这样（你的报告实验会自己复现它）：



函数签名：

```
def dirichlet_energy(X, A):
    # 返回 float >= 0

def propagate(A_norm, X, k):
    # 返回 A_norm^k @ X；k = 0 时返回 X 本身
```

验证：

```
uv run pytest tests/test_graph.py -k "dirichlet_energy or propagate"
```

应看到 2 passed。

检查点 A

```
uv run pytest tests/test_graph.py
```

应看到 14 passed。

Task 2：GCN 图卷积层

现在实现模型的核心积木。一个 GCN 层是两次矩阵乘法：

$$out = \underbrace{\hat{A}}_{\text{传播}} \underbrace{X}_{\text{特征}} \underbrace{W}_{\text{变换}} + b$$

左乘 $\hat{A}$ 完成无参数的消息传播，右乘 W 完成可学习的特征变换。两步都可以直接表示为矩阵乘法，便于实现高效的前向与反向传播。

> 这一整节只允许使用 NumPy，不得调用任何自动微分工具。

步骤 2.1：`aggregate` 与 `aggregate_backward`

目标：把「左乘 $\hat{A}$」单独实现为一个算子。它不带参数，却是图网络区别于 MLP 的全部所在。

前向 $H_{agg} = \hat{A}X$；由链式法则，反向为

$$\frac{\partial L}{\partial X} = \hat{A}^\top \frac{\partial L}{\partial H_{agg}}$$

注意是 $\hat{A}^\top$ 而不是 $\hat{A}$：消息从 j 流向 i（$\hat{A}_{ij}$），梯度就沿同一条边流回 j。只有当 $\hat{A}$ 对称（对称归一化）时两者才恰好相等——`test_aggregate_backward_uses_transpose` 用非对称矩阵专门检查这一点。

函数签名：

```
def aggregate(A_norm, X):
    # 返回 (H_agg: (N, F), cache: (A_norm, X))

def aggregate_backward(dH_agg, cache):
    # 返回 dX: (N, F)
```

验证：

```
uv run pytest tests/test_gcn.py -k aggregate
```

应看到 4 passed。

步骤 2.2：`graph_conv_forward_naive`

目标：用显式双重循环实现图卷积——对每个节点 i，先对邻居做加权和 $\sum_j \hat{A}_{ij} x_j$，再乘 W 加 b。它不追求速度，唯一使命是作为「显然正确」的基准，供矩阵版本比对。只在测试的小图上用它，训练时绝不调用。
函数签名：

```
def graph_conv_forward_naive(A_norm, X, W, b):
    # A_norm: (N, N), X: (N, F_in), W: (F_in, F_out), b: (F_out,)
    # 返回 (out: (N, F_out), cache: (A_norm, X, W, b))
```

验证：

```
uv run pytest tests/test_gcn.py -k naive
```

应看到 1 passed。

步骤 2.3：`graph_conv_forward`

目标：用两次矩阵乘法实现图卷积，结果与朴素版数值完全一致，但快几个数量级。

函数签名：

```
def graph_conv_forward(A_norm, X, W, b):
    # 返回 (out: (N, F_out), cache: (A_norm, X, W, b, H_agg))
```

验证：

```
uv run pytest tests/test_gcn.py -k "matches_naive or shapes or bias"
```

应看到 3 passed。其中 `test_graph_conv_is_faster_than_naive` 会计时对比两个实现。

步骤 2.4：`graph_conv_backward`

目标：实现图卷积的反向传播。前向对 (H_{agg}, W, b) 而言是仿射变换，再把梯度沿 $\hat{A}^\top$ 传播回节点：

$$\partial W = H_{agg}^\top \, \partial out, \qquad \partial b = \textstyle\sum_i (\partial out)_i, \qquad \partial X = \hat{A}^\top (\partial out \, W^\top)$$

函数签名：

```
def graph_conv_backward(dout, cache):
    # dout: (N, F_out)；cache: 来自 graph_conv_forward
    # 返回 (dX, dW, db)
```

验证：

```
uv run pytest tests/test_gcn.py -k "backward or receptive or neighbours"
```

应看到 6 passed。`test_graph_conv_gradient_reaches_neighbours` 检查梯度恰好流到「闭邻域」（自身 + 一跳邻居），`test_two_hop_receptive_field` 把两层串起来，检查梯度的感受野扩大到两跳。

检查点 B

```
uv run pytest tests/test_gcn.py
```

应看到 13 passed。

Task 3：GraphSAGE 与通用聚合器

GCN 把「自己」和「邻居」混在一起加权（靠自环），GraphSAGE[2] 换了一种分解方式——拼接而不是混合：

$$out = \mathrm{concat}\big(X,\ \mathrm{AGG}(X)\big)\,W + b, \qquad W \in \mathbb{R}^{2F_{in}\times F_{out}}$$

自身特征与邻居聚合结果各占 concat 的一半，权重矩阵上下两块可以独立地学习「信自己多少、信邻居多少」。聚合器 AGG 只要求是排列不变的（邻居是一个集合，没有顺序），GraphSAGE 论文给出了几种选择：

- mean：$\mathrm{AGG}(X)_i = \frac{1}{|\mathcal{N}(i)|}\sum_{j\in\mathcal{N}(i)} x_j$——就是左乘 $D^{-1}A$，即 `row_normalize` 后的 `aggregate`，注意这里不加自环（自身信息已经由 concat 负责）；
- max：逐特征通道取邻居的最大值，只保留每个通道上最强的信号。

> 这一整节只允许使用 NumPy，不得调用任何自动微分工具。

步骤 3.1：`max_aggregate` 与 `max_aggregate_backward`

目标：实现 max 聚合器及其反向。前向时，每个节点在每条特征通道上取邻居的最大值，孤立节点得到零向量；反向时，梯度只传给取得最大值的邻居，其余邻居梯度为零（若并列则平分）。

函数签名：

```
def max_aggregate(A, X):
    # A: (N, N) 邻接（只看稀疏模式，无需归一化），X: (N, F)
    # 返回 (H_agg: (N, F), cache: (A, X))

def max_aggregate_backward(dH_agg, cache):
    # 返回 dX: (N, F)
```

实现提示：对每个节点 `i`，`np.flatnonzero(A[i] > 0)` 给出邻居下标，之后 `X[nbrs].max(axis=0)` 即可完成前向。逐节点循环在 Cora 规模（2708 节点）上完全够用。

验证：

```
uv run pytest tests/test_sage.py -k max_aggregate
```

应看到 4 passed。

步骤 3.2：`sage_conv_forward` 与 `sage_conv_backward`

目标：实现 GraphSAGE 卷积层。前向是「聚合 → 拼接 → 仿射」三步；反向把 $\partial\, combined$ 沿拼接缝切成两半：前半是直通自身的梯度，后半走聚合器的反向流回邻居。

函数签名：

```
def sage_conv_forward(A_adj, X, W, b, aggregator="mean"):
    # A_adj: (N, N)——mean 用 row_normalize(A)（无自环），max 用原始 A
    # X: (N, F_in)，W: (2 * F_in, F_out)，b: (F_out,)
    # 返回 (out: (N, F_out), cache)

def sage_conv_backward(dout, cache):
    # 返回 (dX, dW, db)
```

实现要求：`aggregator="mean"` 时聚合必须复用你在 2.1 实现的 `aggregate` / `aggregate_backward`；`aggregator="max"` 时复用 3.1。

验证：

```
uv run pytest tests/test_sage.py
```

应看到 11 passed。其中 `test_sage_conv_equivalent_to_split_weights` 验证 $concat(X, \mathrm{AGG})[W_{top}; W_{bot}] = XW_{top} + \mathrm{AGG}\,W_{bot}$——拼接可以拆成两条独立的通路，这对理解它的反向很有帮助。

检查点 C

```
uv run pytest tests/test_sage.py
```

应看到 11 passed。

Task 4：半监督节点分类模型

积木已经备齐，本任务组装完整模型。先处理一个 GNN 特有的问题：掩码损失。

4.1 半监督掩码损失

节点分类的标准设定是半监督 + 直推式（transductive）：整张图（所有节点的特征 + 全部边）在训练时始终可见，但只有极少数节点的标签允许参与损失计算。Cora 上每类只标注 20 个节点——2708 个节点里只有 140 个进入损失函数。

实现上用一个布尔掩码 `mask` 标记哪些节点有标签。前向照常对全图计算 scores；损失只在 `mask` 为真的行上取 softmax 交叉熵并按有标签节点数（而不是 N）求平均。反向的关键约束是：无标签节点的分数行梯度必须恰好为零——它们影响损失的唯一途径是通过消息传递间接影响有标签节点的表示，这由 `graph_conv_backward` 里的 $\hat{A}^\top$ 自动完成，不需要（也不允许）你手动把梯度传过去。

函数签名：

```
def masked_softmax_loss(scores, y, mask):
    # scores: (N, C)；y: (N,) 整数标签（掩码外的取值忽略）；mask: (N,) bool
    # 返回 (loss: float, dscores: (N, C))，dscores 在 mask 为 False 的行上恰为 0

def masked_accuracy(scores, y, mask):
    # 返回掩码内节点的分类准确率，float
```

实现要求：直接对 `src/layers.py` 中已提供的 `softmax_loss` 做掩码索引即可，不必重写 softmax 本身。

验证：

```
uv run pytest tests/test_model.py -k masked
```

应看到 4 passed。

4.2 组装 `GraphNet` 类

请在 `src/model.py` 中补全 `GraphNet` 类。构造函数 `__init__` 已经写好（这就是那 3 个天然通过的测试中的两个），你只需要补全前向/反向相关的方法。 结构为：

```
[graph-conv - ReLU] × (num_layers - 1)  →  graph-conv  →  scores
```

- 「graph-conv」由 `self.layer_type` 决定：`"gcn"` 调 `graph_conv_forward`（`W` 形状 (F_{in}, F_{out})），`"sage"` 调 `sage_conv_forward`（`W` 形状 $(2F_{in}, F_{out})$）。
- 邻接矩阵不是参数：它在模型外一次性算好（GCN 用 `symmetric_normalize(A + I)`，SAGE-mean 用 `row_normalize(A)`，SAGE-max 用原始 `A`），每次调用时作为 `A_norm` 传入。所有层共享同一个 `A_norm`，因此 L 层网络的每个节点分数依赖于它的 L 跳邻域。
- 参数以 `dict` 存储（`W0/b0/W1/b1/...`），梯度同样以 `dict` 返回，`optim.py` 中的 `sgd` / `sgd_momentum` / `adam` 可以直接逐参数使用，无需修改。
- 权重初始化用 `src/init.py` 中已提供的 `kaiming`。

步骤 4.1：`_conv` / `_conv_backward` / `_conv_relu` / `_conv_relu_backward`

函数签名：

```
def _conv(self, X, A_norm, idx):
    # 第 idx 层图卷积（按 self.layer_type 分派），返回 (out, conv_cache)

def _conv_backward(self, dout, cache):
    # 返回 (dX, dW, db)

def _conv_relu(self, X, A_norm, idx):
    # 一个 graph-conv-relu 块的前向，返回 (out, (conv_cache, relu_cache))

def _conv_relu_backward(self, dout, caches):
    # 逆序反传，返回 (dX, dW, db)
```

步骤 4.2：`loss` 与 `_backward`

函数签名：

```
def loss(self, X, A_norm, y=None, mask=None):
    # X: (N, num_features)；A_norm: (N, N) 预处理好的邻接
    # y=None → 返回 scores (N, num_classes)
    # 否则   → 用 masked_softmax_loss 计算损失，返回 (loss, grads)

def _backward(self, dscores, caches, last_cache):
    # 从 dscores 反传回所有参数的梯度，返回 grads dict（键与 params 一致）
```

步骤 4.3：`predict`

函数签名：

```
def predict(self, X, A_norm):
    # 返回每个节点的预测类别 (N,)
```

验证：

```
uv run pytest tests/test_model.py
```

应看到 12 passed。其中有 4 个数值梯度测试（2 层 GCN、SAGE-mean、SAGE-max、3 层 GCN），任何一个符号错误都会在 `rel_error < 1e-5` 的精度要求下暴露。

检查点 D

```
uv run pytest tests/test_model.py
```

应看到 12 passed。

Task 5：训练循环与 Cora 实战

5.1 训练循环（`src/train.py`）

整张图是一个训练样本，因此不使用 minibatch。每个 epoch 对全图完成一次前向传播、反向传播和参数更新，但损失只由 `train_mask` 选中的节点计算。这种「图全体可见、标签部分可见」的设定称为直推式（transductive）学习。

`TrainConfig` dataclass 已给出（第 3 个天然通过的测试），你只需补全两个函数。

步骤 5.1：`evaluate`

函数签名：

```
def evaluate(net, X, A_norm, y, mask):
    # 返回 (掩码内的平均损失, 掩码内的准确率)
```

步骤 5.2：`train`

函数签名：

```
def train(X, A_norm, y, train_mask, cfg: TrainConfig, val_mask=None, verbose=True):
    # 返回 (net, history)
```

实现要求：

1. 构造 `GraphNet(num_features=X.shape[1], num_classes=int(y.max())+1, hidden_dim=cfg.hidden_dim, num_layers=cfg.num_layers, layer_type=`，并为每个参数初始化一个优化器 config（`{k: {"learning_rate": cfg.lr} for k in net.params}`）。
2. 从 `{"sgd": sgd, "sgd_momentum": sgd_momentum, "adam": adam}` 中按 `cfg.optimizer` 选出更新函数。
3. 主循环：每个 epoch 计算 `(loss, grads) = net.loss(X, A_norm, y, train_mask)`，若 `cfg.weight_decay > 0`，对每个权重（名字以 `W` 开头）的梯度加上 L2 正则项 `cfg.weight_decay * net.params[name]`，再逐参数调用更新函数。
4. 每个 epoch 结束时用 `evaluate` 记录训练掩码上的指标；若给了 `val_mask`，也记录验证掩码指标，存入 `history`。`history` 是一个 dict，含列表 `train_loss`、`train_acc`，以及（有验证集时）`val_loss`、`val_acc`。
5. `verbose=True` 时每 20 个 epoch（以及最后一个 epoch）打印一次进度。

验证：

```
uv run pytest tests/test_train.py
```

应看到 8 passed。最后两个测试值得注意：`test_graph_structure_helps` 在两个集团（clique）构成的合成图上训练，特征噪声大到 MLP（恒等邻接，无消息传递）学不动，而 GCN 借聚合降噪轻松超过 95%——图结构的价值被单独剥离了出来；`test_transductive_unlabeled_nodes_improve` 检查从未进入损失的节点在训练后也被分对——标签沿着边传播了出去。

检查点 E：全部通过

```
uv run pytest
```

应看到 58 passed。至此，你的 GNN 实现已经通过了所有自动化验证。

5.2 数据集与预处理

Cora（见参考资料[3]）是图神经网络最经典的基准数据集：2708 篇机器学习论文为节点，5429 条引用关系为边（本实验视为无向）；每个节点的特征是 1433 维的 0-1 词袋向量（该论文是否包含词表中的每个词），标签是 7 个学科类别之一。加载代码已经提供在 `src/data.py` 中。

> 关于数据集下载：`load_cora` 首次运行时会自动从以下地址下载（约 170 KB）并解压到 `data/cora/`，再解析成 `data/cora.npz` 缓存，之后每次加载直接从缓存读取、瞬时完成：
>
> ```
> https://linqs-data.soe.ucsc.edu/public/lbc/cora.tgz
> ```
>
> 如果你已经手动下载并解压好了这份数据，只要保证 `cora.content` 与 `cora.cites` 位于 `data/cora/`（或 `data/cora/cora/`）即可，脚本会自动检测、跳过下载。加载与划分的用法如下：

```
from src.data import load_cora, make_splits, row_normalize_features

X, y, edges = load_cora("data")                    # (2708,1433) 特征, (2708,) 标签, (5429,2) 边
X = row_normalize_features(X)                      # 词袋 → 词频（行归一化）
train_mask, val_mask, test_mask = make_splits(X.shape[0], y, labels_per_class=20)
```

`make_splits` 采用引用网络的标准协议：每类恰好 `labels_per_class` 个有标签节点进训练掩码，再各取 500 / 1000 个节点进验证/测试掩码，其余节点既不进损失也不进评估。

> 关于稀疏矩阵：Cora 有 2708 个节点却只有 5429 条边——邻接矩阵 99.8% 是零。稠密存储下每层的 $\hat{A}X$ 是 $2708 \times 2708 \times 1433$ 次乘加，慢到无法接受。因此 `train_cora.py` 在归一化之后把邻接矩阵转成 `scipy.sparse.csr_matrix` 再交给模型。由于层实现基于 `@` 与 `.T`，同一份代码可以处理稠密数组和稀疏矩阵，无需修改。测试用例全部在小型稠密矩阵上运行，不依赖 scipy 稀疏格式。

> 注意： `tests/` 下所有测试都在合成小图上运行，不需要下载 Cora 即可通过全部 58 例。你可以先写完代码、跑绿全部测试，再去下载真实数据集做正式训练。

5.3 正式训练与调参（报告内容）

通过全部测试后，在真实的 Cora 数据集上训练节点分类模型（数据加载与训练流程已写好在 `src/data.py`、`train_cora.py`）：

```
uv run python train_cora.py
```

默认配置（2 层 GCN、对称归一化、hidden 16、adam、lr 0.01、200 epochs、每类 20 个标签）每次运行约需一到两分钟，测试准确率通常在 75%~78% 附近（作为对照，恒等邻接的 MLP 基线约为 56%）。

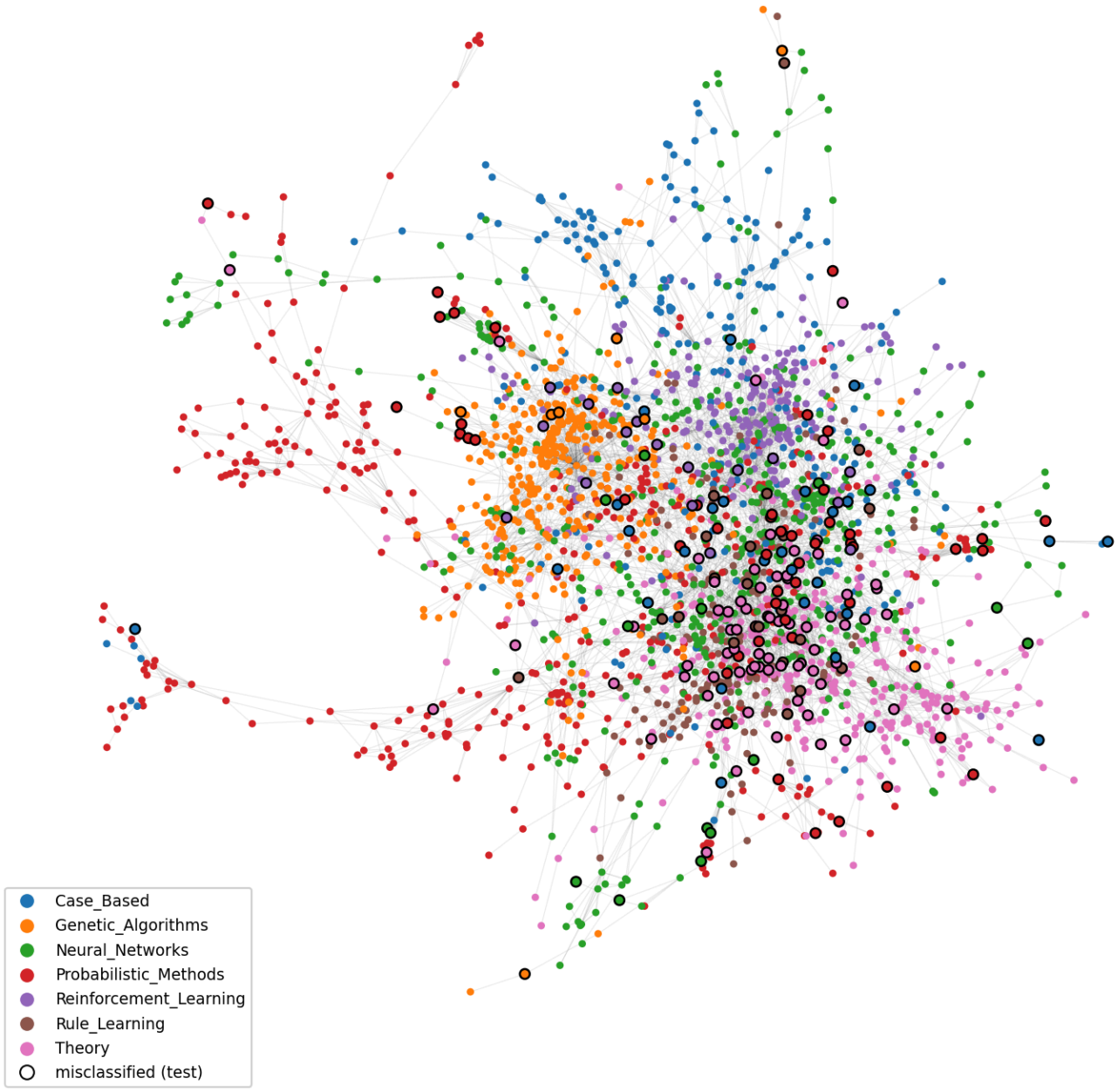
训练脚本会自动生成以下输出，供你查看训练效果并整理实验报告：

1. 损失 / 准确率曲线：保存为 `outputs/loss_curve.png` 与 `outputs/accuracy_curve.png`。
2. 测试集逐类准确率：在 1000 个测试节点上计算总体准确率与每个类别的准确率，并将预测结果保存到 `outputs/test_predictions.npz`（供可视化脚本读取）。

训练结束后，运行：

```
uv run python visualize_graph.py
```

脚本会把 Cora 的最大连通分量用弹簧布局画出来（`outputs/cora_Graph.png`），节点按预测类别着色，预测错误的测试节点用黑圈标出。你会看到颜色明显聚成一簇一簇——同类论文互相引用，这正是 GNN 能工作的前提（同质性假设）。参考运行结果如下：



你在实验报告里面需要写什么?

一、请完成以下三组精简对照实验。每组只比较两个代表配置，每组用几句话说明结果即可。默认配置只需运行一次，可在三组比较中复用，因此共需完成四次训练。

1. 归一化：比较默认配置 `--norm sym` 与不归一化的 `--norm none`，说明归一化对训练稳定性和测试准确率的影响。
2. 层数与过平滑：比较 `--num-layers 2` 与 `--num-layers 4`；另运行一次 `uv run python train_cora.py --smoothness`，结合 Dirichlet 能量的变化解释两者表现差异。
3. 图结构消融：比较默认 GCN 与 `--identity-adj`（恒等邻接，即无消息传递的 MLP），说明图结构是否提升了分类效果。

二、回答以下两个思考题：

1. 层数加深为什么会出现「过平滑」？结合 $\hat{A}^k$ 的变化趋势和测得的 Dirichlet 能量作答。
2. 只给每类 20 个标签时，GCN 为什么通常明显优于不使用图结构的 MLP？图中的边为半监督学习提供了什么信息？

最终报告应包含：四次训练结果的汇总表、三组对照实验的简要分析、两道思考题的回答，以及一张由 `visualize_graph.py` 生成的 `outputs/cora_Graph.png`。无需提交 `test_predictions.npz`，也无需记录完整命令行。实验报告无模版、格式不限。

提供的代码（无需修改）

以下文件已经写好，直接使用：

文件	内容
`src/layers.py`	`affine_forward/backward` 、 `relu_forward/backward` 、 `softmax_loss`
`src/optim.py`	`sgd` / `sgd_momentum` / `adam`
`src/init.py`	`kaiming`：线性层权重的 He 初始化
`src/data.py`	Cora 下载、加载、缓存、特征行归一化与 train/val/test 掩码划分
`tests/gradient_check.py`	数值梯度检验工具
`train_cora.py`	Task 5 的命令行训练脚本（属可修改范围）
`visualize_graph.py`	引用网络可视化脚本

评分标准

本次实验满分 100 分，由「自动化测试（80 分）」+「Cora 训练与报告（20 分）」两部分组成。两部分相互独立、分别打分后相加。

第一部分：自动化测试（80 分）

由 `uv run pytest` 共 58 例的通过比例决定：

$$\text{本部分得分} = \frac{\text{通过用例数}}{58} \times 80$$

其中 3 个测试无需写任何代码即可通过（仅检查已提供的 `TrainConfig` 与 `GraphNet.__init__`）。请假定助教不会逐行阅读你的源代码，仅通过运行测试用例计算这部分分数。但会对代码进行抽查。不得针对测试用例硬编码、或以其他方式绕过计算。

第二部分：Cora 训练与报告（20 分）

这部分考察你是否真正把模型在真实数据上「跑起来、跑对、能看懂」，评分依据为提交的报告 + 运行产物。得分要点：

1. 完成三组精简对照实验并作简要分析（12 分）
2. 回答两道思考题，思路基本正确（6 分）
3. 报告书写规范，结果汇总清晰，并包含生成的图可视化（2 分）

提交要求

请按要求完成实验代码，之后：

1. 将根目录下的 `.venv` 、 `.pytest_cache` 、 `data` 、 `outputs` 目录删除；
2. 将代码根目录和实验报告以 zip 格式打包提交，压缩包命名为 `学号-姓名` 。

注意以下几点：

- 实验过程中，除了 `uv` 自动生成的文件，你只能修改 `src` 下的源文件（ `train_cora.py` 为本次提供的运行脚本，属可修改范围）；
- 第一部分的自动化测试得分仅由 `uv run pytest` 决定；第二部分的 20 分由助教依据上述标准人工评定。

提交 ddl：2026 年 11 月 12 日。

参考资料

[1] Kipf, T. N., & Welling, M. (2017). Semi-Supervised Classification with Graph Convolutional Networks. *ICLR*. https://arxiv.org/abs/1609.02907 GCN 的论文。本实验的层公式 $\hat{A}HW$（含重归一化技巧 $\tilde{A} = A + I$、$\hat{A} = \tilde{D}^{-1/2}\tilde{A}\tilde{D}^{-1/2}$）与 Cora 半监督设定均来自此文。

[2] Hamilton, W. L., Ying, R., & Leskovec, J. (2017). Inductive Representation Learning on Large Graphs. *NeurIPS*. https://arxiv.org/abs/1706.02216 GraphSAGE 的论文。concat 自身特征 + 排列不变聚合器（mean / max / LSTM）的设计来自此文第 3 节。

[3] Sen, P., Namata, G., Bilgic, M., Getoor, L., Gallagher, B., & Eliassi-Rad, T. (2008). Collective Classification in Network Data. *AI Magazine*, 29(3), 93–106. https://linqs.soe.ucsc.edu/data Cora 数据集与「利用网络结构做集体分类」的早期系统研究。

[4] McCallum, A., Nigam, K., Rennie, J., & Seymore, K. (2000). Automating the Construction of Internet Portals with Machine Learning. *Information Retrieval*, 3(2), 127–163. Cora 数据集的原始出处。

[5] Li, Q., Han, Z., & Wu, X.-M. (2018). Deeper Insights into Graph Convolutional Networks for Semi-Supervised Learning. *AAAI*. https://arxiv.org/abs/1801.07606 系统分析了过平滑问题：GCN 的聚合是拉普拉斯平滑的特例，层数加深会使节点特征趋于相同。对应本实验的思考题 2。

[6] Gilmer, J., Schoenholz, S. S., Riley, P. F., Vinyals, O., & Dahl, G. E. (2017). Neural Message Passing for Quantum Chemistry. *ICML*. https://arxiv.org/abs/1704.01212 消息传递框架（MPNN）：把 GCN、GraphSAGE、GAT 等统一表述为「消息函数 + 聚合 + 更新」。本实验「聚合 / 变换」两步拆分即沿用此视角。

[7] Veličković, P., Cucurull, G., Casanova, A., Romero, A., Liò, P., & Bengio, Y. (2018). Graph Attention Networks. *ICLR*. https://arxiv.org/abs/1710.10903 GAT：让每条边的权重由注意力机制学习，而不是由度数预先决定。本实验不涉及，供有兴趣的同学延伸阅读。

[8] Wu, F., Souza, A., Zhang, T., Fifty, C., Yu, T., & Weinberger, K. (2019). Simplifying Graph Convolutional Networks. *ICML*. https://arxiv.org/abs/1902.07153 SGC：去掉 GCN 的所有非线性后仅剩 $\hat{A}^k X W$，性能却几乎不变——说明 GCN 的主要能力来自图上的低通滤波。本实验的 `propagate` 函数就是它的「无权重版本」。

[9] Zhou, J., Cui, G., Hu, S., Zhang, Z., Yang, C., Liu, Z., Wang, L., Li, C., & Sun, M. (2020). Graph Neural Networks: A Review of Methods and Applications. *AI Open*, 1, 57–81. https://arxiv.org/abs/1812.08434 GNN 综述，系统整理了消息传递框架下各种变体的分类。

[10] Shuman, D. I., Narang, S. K., Frossard, P., Ortega, A., & Vandergheynst, P. (2013). The Emerging Field of Signal Processing on Graphs. *IEEE Signal Processing Magazine*, 30(3), 83–98. https://arxiv.org/abs/1211.0053 图信号处理综述：图拉普拉斯、图上傅里叶变换与 Dirichlet 能量的来龙去脉，对应 Task 1.5/1.6。